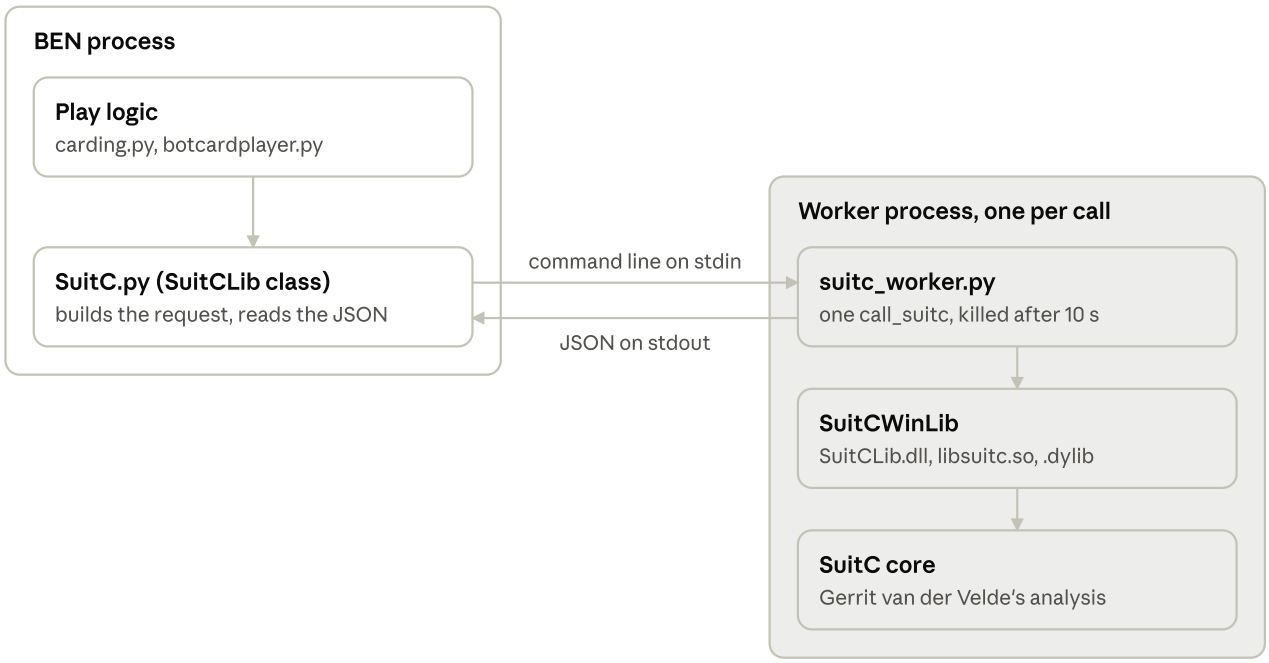
Overview

BEN uses SuitC as a second opinion in declarer play. When declarer is about to play a suit for the first time, BEN asks SuitC for the best way to play that suit on its own. It plays SuitC's card if that card is also among the cards its double-dummy simulation rates best. BEN also uses SuitC to estimate how many tricks each suit is worth when it compares lines of play.

BEN never links the SuitC C code directly. Four layers sit between BEN's play logic and Gerrit's analysis:

1. **BEN's play logic**: `src/carding.py` (choosing a card) and `src/botcardplayer.py` (trick estimates).
2. **`src/suitc/SuitC.py`**: the `SuitCLib` class, which builds the SuitC command line and reads the JSON result.
3. **`src/suitc/suitc_worker.py`**: a short-lived Python process that loads the native library and makes one call, so BEN can kill a call that hangs.
4. **SuitCWinLib** (`SuitCLib.dll`, `libsuitc.so`, `libsuitc.dylib` in `bin/`): Lars Kappel Hansen's library build of SuitC, which takes a SuitC command line and returns JSON. Inside it runs Gerrit van der Velde's SuitC core.

Each SuitC call runs in its own short-lived process



BEN never loads the library into its own process: a hung analysis can only take down the worker.

SuitC is optional. It is switched on per configuration with `use_suitc = True`, and BEN falls back to its double-dummy choice whenever SuitC is off, fails or has no suggestion.

Loading the library

BEN loads one native file from its `bin/` folder, chosen by platform. The folder is found next to the sources when run from `src/`, otherwise as `$BEN_HOME/bin` or `./bin`.

Platform	File in `bin/`	Built for
Windows	`SuitCLib.dll`	x64
Linux	`libsuitc.so`	x64
macOS	`libsuitc.dylib`	arm64

At start-up, `game.py`, `gameapi.py` and `gameserver.py` create a `SuitCLib` when `use_suitc` is on and print `SuitC enabled. Version <n>` from the library's `version()` call. `gameserver.py` also turns SuitC off if the library fails to load; the other two stop with an error naming the file.

The analysis itself does not run in BEN's own process. Each call starts `suitc_worker.py` as a child process. The worker loads the library with `ctypes`, calls `call_suitc` once and prints one JSON object (`rc`, `output`, `details`). In a PyInstaller build there is no Python or worker script on disk, so BEN starts its own executable again with `--suitc-worker`, and the entry point hands that to the worker before loading anything else.

The worker keeps file descriptor 1 pointed at `/dev/null` for its whole life, because the library still prints its game tree to standard output. Its JSON goes to a saved copy of the real standard output.

When BEN calls SuitC

BEN calls SuitC in two places, both only for the declaring side.

Choosing the card to lead in a new suit

`select_right_card_for_play` in `src/carding.py` runs after BEN's simulation has ranked the candidate cards. It asks SuitC only when all of these hold:

1. `use_suitc = True` in the configuration's `[cardplay]` section.
2. The simulation has no clear winner. Unless `force_suitc = True`, BEN plays its top card straight away when it beats the second card by more than 0.1 in chance of making, 1 double-dummy trick, 5 matchpoints or 1 IMP.
3. Declarer or dummy is on lead (`play_status == "Lead"`).
4. The contract is notrump, the suit is trumps, or `suitc_sidesuit_check = True` allows

side suits too.

5. The suit has not been led yet. At most two of the leader's cards in it are gone, and fewer than three cards of it have been played by the other hands.
6. The two hands together hold at least six cards in the suit.
7. The opponents hold at least two cards in the suit, and at most six unless it is the trump suit.

Estimating tricks in notrump

`calculate_suit_adjust_for_nt` in `src/botcardplayer.py` adjusts the candidates' scores per suit when the declaring side is on lead in notrump and `use_suit_adjust = True`. For every suit where the longer hand has at least two cards and the opponents hold some, it asks SuitC for the expected number of tricks (`get_suit_tricks`) and compares that with the length of the longer hand.

Configuration

Setting (`[cardplay]`)	Default	Effect	Configs that set it True
`use_suitc`	False	Loads SuitC and allows both uses above	19 of 22 that mention it
`force_suitc`	False	Asks SuitC even when the simulation has a clear winner, and accepts a bigger loss against the best card	14
`suitc_sidesuit_check`	False	Also asks SuitC for side suits in a trump contract	21
`use_suit_adjust`	False	Turns on the notrump trick estimates	14

Building the request

BEN sends SuitC an ordinary SuitC command line: options, then the North, South and East-West holdings in one suit. An empty holding is sent as `.`.

Choosing a card: `SuitCLib.calculate`

- **North** is the hand on lead, declarer or dummy. **South** is its partner.
- **East-West** is every card of the suit that is in neither hand and has not been played yet.
- **Options** are always `-F1 -u -c100`, plus `-ls<n>` in notrump and side suits. `-a -b` are added in verbose mode.

Option	Meaning in SuitC	Why BEN sets it

`-F1`	Fold the East-West plays in the game tree	One line per North-South play, so the first card is easy to read
`-u`	Among equal plays, win tricks early and use the fewest outside entries	Keeps entries for the rest of the hand
`-c100`	Cutoff while merging quirk trees; SuitC raises it to at least 20000 (Rev 2.020 and later)	Keeps each call fast
`-ls<n>`	South can be reached from outside the suit at most `<n>` times	Models the entries the side really has

`<n>` comes from `count_entries`. It counts the unplayed outside honours of the hand on lead (ace and king 1, queen ½), counts every unplayed trump as one, and rounds the total. No `-ls` is sent when the suit is trumps. BEN never sends `-M` or `-I`, so SuitC always plays for the maximum number of tricks.

Open question: the entries are counted in the hand on lead (North), but `-ls` limits entries to South. This was consistent before North and South were swapped to make the leader North, and should be checked.

Estimating tricks: `SuitCLib.get_suit_tricks`

The holdings are the two hands of the declaring side and the opponents' remaining cards, with no options. SuitC then tries leads from both hands and reports the expected number of tricks.

Using the result

SuitC suggests; BEN's simulation decides. A SuitC card is played only if it is also one of the simulation's candidates and does not cost too much against the best of them.

Which cards BEN takes from SuitC

`calculate` reads `SuitCAnalysis.OptimumPlays` from the JSON output. For every optimum play whose `OptimumPlayFor` includes `MAX`, it takes the first card North leads in the `GameTree` (a node such as `"T":"N:Q"`). There can be more than one such card. A card that is not in North's holding is ignored, and an empty game tree gives an empty list.

How BEN chooses between SuitC and the simulation

BEN goes through its candidates from best to worst. When the double-dummy-optimal set is known, it only considers cards in that set. A candidate that matches a SuitC card is played if it is no more than 0.15 worse in chance of making the contract and within these margins of the best candidate:

Scoring	Played as	Margin	With `force_suitc`
Matchpoints	`SuitC-MP`	4 MP	up to 40 MP (`SuitC-MP-Forced`); otherwise still played as `SuitC-MP-Worse`
IMPs	`SuitC-Imp`	0.4 IMP	up to 5 IMP (`SuitC-Imp-Forced`)
Tricks, double dummy	`SuitC-DD`	0.2 tricks, and 0.1 in chance of making	none
Tricks, single dummy	`SuitC-SD`	0.2 tricks, and 0.1 in chance of making	none

The label is the reason BEN reports for the card. Forced and worse choices are also logged with `save_for_suitc`, so they can be reviewed. At matchpoints a matching SuitC card is always played, even outside the margins.

If no candidate matches, BEN plays the simulation's best card. If SuitC returns no card at all, BEN plays the best candidate in that suit if it is close to the best card overall (20 MP, 0.2 IMP or 0.2 tricks), assuming the suit is better started from the other hand.

Trick estimates in notrump

`get_suit_tricks` reads `SuitCAnalysis.Result`, the expected number of tricks, rounded to two decimals. For each suit, BEN turns it into an adjustment:

```
adjust = (length of the longer hand - SuitC tricks) / 10
```

If the opponents hold more cards than the longer hand, half their length is subtracted first. The adjustment is added to every candidate in that suit: to its double-dummy tricks and its IMP score, or 20 times the adjustment to its matchpoint score.

Limits and error handling

A SuitC failure never stops a deal. Every error ends in BEN's own choice of card, or in no adjustment for that suit.

Problem	What `SuitC.py` does	What the caller does
Call runs longer than 10 s	Kills the worker and raises `SuitCTimeout`, naming the holding	Card choice: plays the simulation's best card. Trick estimate: adjustment 0 for that suit
Worker exits with an error, or prints something that is not JSON	Raises `RuntimeError` with the worker's output	Same as above

SuitC rejects the command line (`rc` not 0)	Raises `RuntimeError` with SuitC's error text	Same as above
Library missing or blocked at start-up	Prints where it looked and raises `RuntimeError`	`gameserver.py` turns SuitC off; `game.py` and `gameapi.py` stop

The 10-second limit can be changed with the environment variable `SUITC_TIMEOUT_SECONDS`. The separate process exists because a runaway loop in SuitC's quirk-tree merge once froze BEN's single-threaded game server; a process can be killed, a call inside BEN cannot.

The price is one process start and one library load per call. BEN keeps the number of calls down: at most one per suit, the first time the suit is led, and only when the simulation has no clear winner.

The size of a call is limited too. BEN skips suits where the opponents hold more than six cards outside trumps. The library, built with `SUITCAPP=YES`, allocates its node and quirk pools inside the worker on each call, and the worker's output buffers hold 262,144 characters, twice what SuitCWinLib needs.

Updating the library

BEN's three libraries come from the releases of SuitCWinLib on GitHub (`codescrut/SuitCWinLib`). Each release is built and tested by CI on every platform, and its version follows the library's `version()` call. As of 18 October 2026, BEN's committed libraries are 0.9.1.0. Version 1.0.0.1, with SuitC Rev 2.022, is being installed.

Version	Date	SuitC core
1.0.0.1	2026-10-18	Rev 2.022: throw-in removed again, `-I` checked
1.0.0.0	2026-10-17	Rev 2.020
0.9.1.0	2026-10-10	Rev 2.011

To update BEN:

1. Download from the release `suitclib-v<version>-windows-x64.zip`, `suitclib-v<version>-linux-x64.tar.gz` and `suitclib-v<version>-macos-arm64.tar.gz`.
2. Replace `bin/SuitCLib.dll`, `bin/libsuitc.so` and `bin/libsuitc.dylib` with the files inside. Stop any running BEN first; Windows locks a DLL in use.
3. Start BEN with a configuration that has `use_suitc = True`, and check that it prints `SuitC enabled. Version <version>`.
4. Run `python test.py` in `src/suitc` for a quick call through the worker.